Fundamentos de Inteligencia Artificial para Ingenieros de Software

Tema 3. Automatización de tareas repetitivas en desarrollo

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Identificación de tareas repetitivas en desarrollo

3.3. Desarrollo de scripts automatizados

3.4. Integración de automatización en flujos de trabajo

3.5. Herramientas populares de automatización en desarrollo

3.6. Mejora de eficiencia y reducción de errores con automatización

3.7. Referencias bibliográficas

A fondo

Centro de aprendizaje DataDog

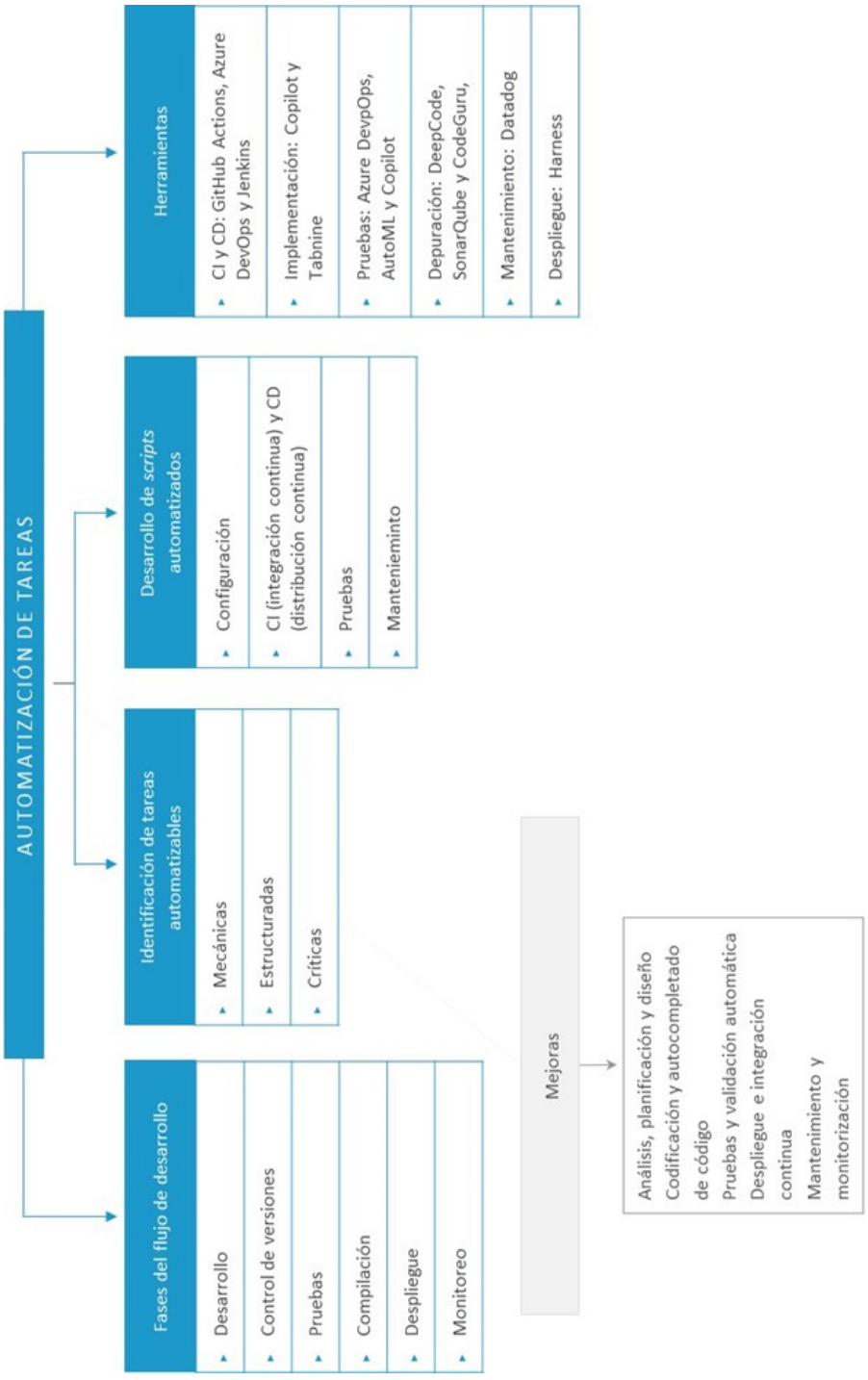
Guías para GitHub Actions

Snyk Code

From zero to hero

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

3.1. Introducción y objetivos

L a **automatización de tareas repetitivas** en el desarrollo de *software* es una tendencia que transformó la industria, ya que les permitió a los desarrolladores **optimizar procesos** que anteriormente consumían tiempo y eran susceptibles a errores humanos.

Las herramientas y técnicas de automatización se integraron profundamente en los flujos de trabajo a través de la integración continua (CI) y el despliegue continuo (CD), lo que mejora la eficiencia y la calidad del *software* entregado.

Esta automatización incluye actividades como la **compilación y construcción del código,** la ejecución de pruebas unitarias y de integración, el **manejo de control de versiones,** la implementación de *software* y el mantenimiento de dependencias. La implementación de ***scripts* automatizados** facilitó que los desarrolladores configuren entornos, realicen pruebas o desplieguen aplicaciones sin intervención manual, lo que contribuye a la mejora de la productividad y a la reducción de errores.

Además, la automatización se extiende a **tareas más avanzadas,** como el monitoreo y la actualización de sistemas en tiempo real, por lo que utiliza herramientas (como Jenkins, Docker, Kubernetes y Terraform) que proporcionan una base sólida para la **automatización eficiente** de procesos en el desarrollo de *software.*

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Los objetivos de este tema son:

- Comprender las técnicas para identificar tareas repetitivas en el ciclo de vida del *software.*
- Aplicar técnicas y herramientas de automatización en el desarrollo de *software.*
- Explorar herramientas populares de automatización y su integración en flujos de trabajo.
- Evaluar la mejora de eficiencia y reducción de errores a través de la automatización.

© Universidad Internacional de La Rioja (UNIR)

3.2. Identificación de tareas repetitivas en desarrollo

La identificación de tareas repetitivas en el desarrollo de *software* es un paso fundamental para optimizar procesos, reducir errores humanos y aumentar la eficiencia mediante la automatización.

A menudo, las tareas que se repiten a lo largo de un proyecto son candidatas ideales para ser **automatizadas,** lo que les permite a los desarrolladores centrarse en **actividades más creativas y de alto valor.**

En el contexto del desarrollo de *software,* las tareas repetitivas son aquellas acciones o procesos que deben ejecutarse de manera regular o bajo ciertas condiciones, ya que **consumen tiempo y recursos** sin aportar un valor estratégico significativo. Estas tareas suelen ser:

- Manuales o mecánicas.
- Predecibles y estructuradas.
- Propensas a errores humanos si no se realizan correctamente.

© Universidad Internacional de La Rioja (UNIR)

Tareas repetitivas comunes en el desarrollo de software

Existen diversas tareas repetitivas a lo largo del ciclo de vida del *software* que pueden ser **identificadas y automatizadas.**

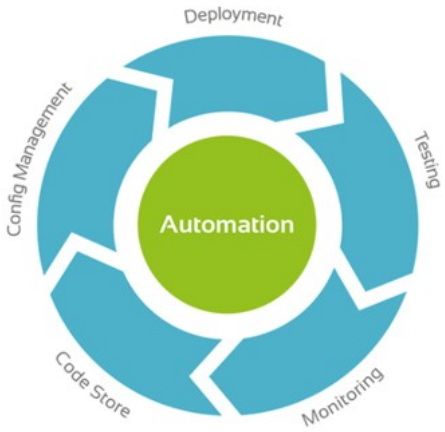


Figura 1. Tareas que se pueden automatizar en desarrollo *software.* Fuente: Bryxx, s. f.

- **Compilación y construcción del código.** El proceso de compilar y construir el código para generar ejecutables o versiones distribuidas del *software* es una tarea frecuente, especialmente en proyectos grandes. Cada vez que se hacen cambios significativos en el código, es necesario reconstruirlo.

- **Pruebas unitarias y de integración.** Las pruebas unitarias verifican la funcionalidad de componentes individuales del código, mientras que las pruebas de integración evalúan cómo interactúan esos componentes. Ejecutar estas pruebas manualmente cada vez que se modifica el código es ineficiente, especialmente en proyectos con cientos o miles de casos de prueba.

- **Control de versiones y manejo de *branches.*** El uso de sistemas de control de versiones como Git requiere la creación, la fusión y el manejo de ramas de código. Las tareas como la creación de nuevas ramas, la resolución de conflictos y la fusión de código son operaciones repetitivas en el ciclo de desarrollo.

© Universidad Internacional de La Rioja (UNIR)

- **Implementación y despliegue de *software.*** Cada vez que el *software* está listo para ser desplegado en un entorno de producción o pruebas, el equipo de desarrollo tiene que seguir un conjunto de pasos predefinidos que incluyen la configuración del servidor, la actualización de bases de datos y la publicación del código. Realizar estos despliegues manualmente puede ser tedioso y sujeto a errores.

- **Mantenimiento de dependencias.** Mantener y actualizar las bibliotecas o dependencias externas de un proyecto es otra tarea que suele repetirse, ya que las actualizaciones periódicas son necesarias para mejorar la seguridad o el rendimiento del *software.*

- **Formateo y estandarización del código.** Seguir convenciones de estilo y formatear el código correctamente es una tarea que se repite a lo largo del desarrollo. Aunque existen reglas definidas, hacer cumplir estas normas manualmente requiere tiempo y es susceptible a inconsistencias.

Métodos para identificar tareas repetitivas

El primer paso para automatizar un proceso es **identificar aquellas tareas que son repetitivas.** Los desarrolladores y equipos pueden utilizar **diversas estrategias** para este fin:

- **Análisis de actividades cotidianas.** Registrar las actividades que los desarrolladores realizan de manera rutinaria puede ayudar a identificar patrones de tareas repetitivas.

  - Por ejemplo, si cada vez que se añade una nueva función al código se ejecutan manualmente una serie de pruebas, esta tarea es un candidato para la automatización.

- **Detección de cuellos de botella.** Las tareas repetitivas suelen estar asociadas con cuellos de botella en los flujos de trabajo. Si un equipo identifica que un paso específico del ciclo de desarrollo toma demasiado tiempo debido a acciones repetitivas, esa tarea puede ser un buen objetivo para la automatización.

© Universidad Internacional de La Rioja (UNIR)

- **Uso de herramientas de monitorización de flujos.** Existen herramientas de monitorización que permiten analizar cómo se ejecutan los flujos de trabajo y detectar qué tareas son las más repetidas o consumen más tiempo. Herramientas como Jenkins, Travis CI o GitLab CI/CD, además de facilitar la automatización, proporcionan informes sobre el número de veces que se ejecutan determinadas tareas.

- **Entrevistas y encuestas internas.** Las entrevistas con los miembros del equipo pueden proporcionar información valiosa sobre qué tareas consideran más tediosas o repetitivas. A menudo, los desarrolladores ya tienen identificado qué parte de su flujo de trabajo podría beneficiarse de la automatización.

Ejemplo práctico de identificación de tareas repetitivas

Los equipos de desarrollo realizan **revisiones del código** para detectar errores y mejorar la calidad del *software.* Si las mismas reglas o patrones son verificados en cada revisión, esto indica que la **tarea es repetitiva** y que se puede automatizar con herramientas como ESLint (para JavaScript) o Pylint (para Python), ya que **verifican automáticamente** la adherencia al estilo de codificación.

En este caso práctico, vamos a utilizar Pylint para revisar automáticamente la adherencia de un proyecto de *software* a un conjunto de reglas de **estilo de codificación** (PEP 8) y principios de buenas prácticas.

PEP 8 *(Python enhancement proposal 8)* es una guía de estilo para escribir código Python de forma clara, legible y coherente.

© Universidad Internacional de La Rioja (UNIR)

Preparación del entorno

Instalación de Pylint

Ejecutar el siguiente comando para instalar Pylint:

```
$> pip install pylint
```

Archivos de Código a Revisar

Se dispone del siguiente archivo de ejemplo llamado gestion_inventario.py:

```
# gestion_inventario.py

class Producto:

    def __init__(self, nombre, precio, cantidad):

        self.nombre = nombre

        self.PRECIO = precio # Nombre en mayúsculas incorrecto para una
variable de instancia

        self.cantidad = cantidad

    def calcular_total(self):

        total = self.PRECIO * self.cantidad

        return total

    def mostrar_detalle(self):

    print(f"Producto: {self.nombre}, Precio: {self.PRECIO}, Cantidad:
{self.cantidad}")
```


© Universidad Internacional de La Rioja (UNIR)

Ejecución de Pylint

Para analizar el archivo gestion_inventario.py, se ejecuta el siguiente comando:

```
$> pylint gestion_inventario.py

Salida de Pylint (Resumen):

************* Module gestion_inventario

gestion_inventario.py:1:0: C0114: Missing module docstring (missing-module-
docstring)

gestion_inventario.py:2:0: C0115: Missing class docstring (missing-class-
docstring)

gestion_inventario.py:4:8: C0103: Attribute name "PRECIO" doesn't conform
to snake_case naming style (invalid-name)

gestion_inventario.py:9:4: C0103: Method name "mostrar_detalle" doesn't
conform to snake_case naming style (invalid-name)

gestion_inventario.py:10:4: E1120: No value for argument 'self' in method
call (no-value-for-parameter)

gestion_inventario.py:11:0: W0311: Bad indentation. Found 1 space, expected
4 (bad-indentation)

-------------------------------------------------------------------

Your code has been rated at **4.00/10** (previous run: 4.00/10, +0.00)
```


© Universidad Internacional de La Rioja (UNIR)

Análisis de los errores y problemas detectados

Tipo de error	Descripción	Línea	Corrección propuesta
C0114 (missing-module-docstring)	Falta la descripción del módulo	1	Añadir una docstring que explique el propósito del archivo
C0115 (missing-class-docstring)	Falta la docstring de la clase	2	Incluir una descripción de la clase Producto
C0103 (invalid-name)	PRECIO no sigue la convención snake_case	4	Cambiar PRECIO por precio
C0103 (invalid-name)	Nombre de método mostrar_detalle incorrecto	9	Cambiar mostrar_detalle a mostrar_detalle_producto
E1120 (no-value-for-parameter)	Método mostrar_detalle no tiene correctamente definido self	10	Ajustar la indentación de la función
W0311 (bad-indentation)	Error de indentación	11	Corregir la indentación para usar cuatro espacios

Tabla 1. Análisis de los errores y problemas detectados. Fuente: elaboración propia.

Corrección del código

```
Archivo Corregido: gestion_inventario.py

"""

Este módulo gestiona la información de productos de un sistema de
inventario.

Incluye la clase Producto que permite calcular el precio total y mostrar
detalles del producto.

"""

class Producto:
```


© Universidad Internacional de La Rioja (UNIR)

```
    """Clase que representa un producto en el inventario."""

    def __init__(self, nombre, precio, cantidad):

        """Inicializa los atributos del producto."""

        self.nombre = nombre

        self.precio = precio # Corregido: nombre de atributo en snake_case

        self.cantidad = cantidad

    def calcular_total(self):

        """Calcula el costo total del producto."""

        total = self.precio * self.cantidad

        return total

    def mostrar_detalle_producto(self):

        """Muestra los detalles del producto en la consola."""

        print(f"Producto: {self.nombre}, Precio: {self.precio}, Cantidad:
{self.cantidad}")
```

Validación de la corrección con Pylint

Se vuelve a ejecutar el análisis con:

```
$> pylint gestion_inventario.py

Salida de Pylint (Resumen):

-------------------------------------------------------------------

Your code has been rated at **10.00/10** (previous run: 4.00/10, +6.00)
```


© Universidad Internacional de La Rioja (UNIR)

Explicación de la mejora:

- Una puntuación de 4.00 a 10.00 indica que el código ahora sigue completamente las reglas de estilo del *PEP 8*.
- Todos los errores fueron corregidos, incluidas las docstrings faltantes, los nombres de variables y los problemas de indentación.

Automatización del proceso con CI y CD

Para automatizar la revisión de estilo de codificación, se puede integrar Pylint en el flujo de CI y CD. A continuación, se muestra un ejemplo con GitHub Actions:

Archivo .github/workflows/pylint.yml:

```
name: Lint Code with Pylint

on:

  push:

    branches:

      - main

  pull_request:

    branches:

      - main

jobs:

  pylint:

    runs-on: ubuntu-latest

    steps:
```


© Universidad Internacional de La Rioja (UNIR)

```
- name: Checkout code
  uses: actions/checkout@v2
- name: Set up Python
  uses: actions/setup-python@v2
  with:
    python-version: '3.8'
- name: Install dependencies
  run: |
    pip install pylint
- name: Run Pylint
  run: |
    pylint gestion_inventario.py
```

Conclusiones

- Pylint mejora la calidad del código, ya que detecta errores comunes de sintaxis, formato y errores lógicos.
- Automatización con CI y CD, puesto que integrar Pylint en CI y CD asegura que las nuevas incorporaciones de código sigan el estándar de estilo.
- Mantenibilidad y consistencia, dado que seguir *PEP 8* con Pylint garantiza que todo el equipo de desarrollo trabaje con las mismas reglas de estilo.

© Universidad Internacional de La Rioja (UNIR)

- Reducción de errores humanos, porque los errores de indentación y nombres incorrectos son comunes, pero Pylint los detecta automáticamente.

Ventajas de la identificación y automatización de tareas repetitivas

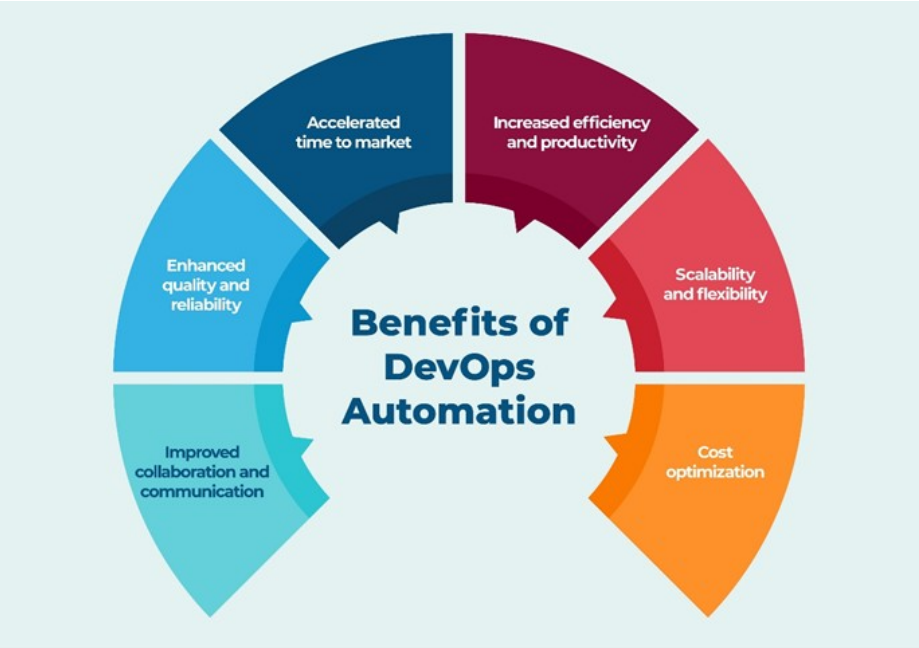


Figura 2. Ventajas de la automatización de tareas en DevOps. Fuente: BuzzyBrains, s. f.

- **Reducción de errores humanos.** Automatizar tareas repetitivas minimiza la probabilidad de errores humanos, que suelen ocurrir cuando las personas realizan las mismas acciones una y otra vez. Por ejemplo, la ejecución manual de *scripts* de despliegue puede fallar debido a una configuración incorrecta, mientras que una tarea automatizada sigue siempre los mismos pasos definidos.

- **Mejora de la productividad.** Al liberar a los desarrolladores de tareas mecánicas, se pueden enfocar en actividades más importantes y creativas, lo que mejora la productividad del equipo en general. Las tareas automatizadas, como las pruebas o la construcción de código, pueden ejecutarse en segundo plano, mientras que los desarrolladores continúan con otras tareas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Ahorro de tiempo.** Al automatizar tareas repetitivas, los tiempos de ciclo de desarrollo se reducen significativamente. Por ejemplo, las pruebas automatizadas pueden ejecutarse en paralelo y mucho más rápido que si se realizan manualmente, lo que permite que el equipo entregue código de mayor calidad en menor tiempo.

© Universidad Internacional de La Rioja (UNIR)

3.3. Desarrollo de scripts automatizados

El desarrollo de *scripts* automatizados mediante herramientas de inteligencia artificial (IA) es un campo en rápida expansión que les permite a los desarrolladores de *software* **crear, optimizar y mantener *scripts*** de manera más eficiente. La IA en la creación de *scripts* no solo **acelera el desarrollo,** sino que también mejora la precisión, **reduce los errores** y permite que los desarrolladores se enfoquen en tareas más complejas.

Los modelos actuales **ofrecen grandes beneficios,** aunque todavía requieren de supervisión humana. La automatización de *scripts* con IA está transformando el desarrollo de *software,* lo que proporciona herramientas que permiten **acelerar los procesos y mejorar la precisión,** así como **impulsar la eficiencia** en el trabajo de los desarrolladores.

Scripts automatizados en desarrollo de software

Un *script* automatizado es un **programa** o conjunto de instrucciones escritas en un **lenguaje de *scripting*** (como Bash, Python, PowerShell, etc.) que ejecuta una **serie de comandos** de manera secuencial o condicional para realizar tareas.

Los *scripts* pueden variar desde simples comandos repetitivos hasta procesos complejos de automatización.

© Universidad Internacional de La Rioja (UNIR)

Los beneficios de la automatización son:

- Eficiencia, ya que los *scripts* automatizados reducen el tiempo requerido para tareas repetitivas.
- Reducción de errores, puesto que minimiza errores humanos, ya que las tareas se ejecutan de manera predefinida.
- Productividad, dado que permite que los desarrolladores se centren en tareas más creativas y menos repetitivas.

Aplicación de la IA en el desarrollo de scripts

La IA se aplica en el desarrollo de *scripts* automatizados en varias etapas del ciclo de vida de estos, desde su creación hasta su optimización y mantenimiento.

- En la optimización de *scripts* existentes, la IA analiza *scripts* antiguos y propone mejoras, como la eliminación de redundancias, la optimización de bucles o la mejora en la gestión de recursos.
- En cuanto a la refactorización automática, hay modelos de IA avanzados, como GPT-4-Turbo y CodeT5, que pueden sugerir una reorganización de *scripts* complejos para mejorar la claridad, la eficiencia y la capacidad de mantenimiento.
- Para la automatización de flujos de trabajo complejos, hay modelos, como Llama 3, que se pueden usar para crear flujos de trabajo complejos que interactúan con múltiples servicios, bases de datos y API *(application programming interface)*, lo que facilita la gestión de sistemas interconectados.

© Universidad Internacional de La Rioja (UNIR)

Ejemplo práctico de scripts automatizados con IA

Automatización de despliegues con DevOps

En DevOps, los despliegues se realizan con frecuencia y requieren *scripts* para coordinar el proceso de lanzamiento, incluyendo pruebas, compilación y publicación.

En este caso, vamos a usar GitHub Copilot para generar un *script* de CI y CD que compile el código, ejecute pruebas unitarias y despliegue la aplicación en un servidor de pruebas. Este incluye *scripts* en YAML *(yet another markup language)* para integrarse con plataformas como GitHub Actions.

Escenario

El equipo de desarrollo trabaja en una aplicación web escrita en Node.js y se necesita un flujo de CI y CD automatizado que:

- Compile el código (si aplica, en este caso no hay compilación explícita para Node.js).
- Ejecute pruebas unitarias mediante Jest .
- Despliegue la aplicación en un servidor de pruebas (a través del protocolo SSH o *secure Shell* para copiar archivos).

Los requisitos son:

- Automatizar el flujo CI y CD con GitHub Actions.
- Asegurar que las pruebas se ejecuten antes del despliegue.
- Implementar un sistema de notificaciones en caso de error.

© Universidad Internacional de La Rioja (UNIR)

Configuración del entorno

Proyecto base y estructura del proyecto:

```
project/
├── src/
│    └── app.js      # Archivo principal de la aplicación
├── tests/
│    └── app.test.js # Pruebas unitarias para la aplicación
├── package.json     # Archivo de configuración de Node.js
└── .github/
     └── workflows/
         └── ci_cd.yml # Archivo de configuración del flujo CI/CD
```

Archivo package.json (Dependencias y Scripts)

```
{
  "name": "ci-cd-example",
  "version": "1.0.0",
  "main": "src/app.js",
  "scripts": {
    "test": "jest",
    "start": "node src/app.js"
```


© Universidad Internacional de La Rioja (UNIR)

```
  },
  "dependencies": {},
  "devDependencies": {
    "jest": "^27.0.0"
  }
}
```

Archivo src/app.js

```
function suma(a, b) {
    return a + b;
}
module.exports = suma;
```

Archivo tests/app.test.js

```
const suma = require('../src/app');
test('suma de 1 + 2 es 3', () => {
    expect(suma(1, 2)).toBe(3);
});
test('suma de 5 + 5 es 10', () => {
    expect(suma(5, 5)).toBe(10);
```


© Universidad Internacional de La Rioja (UNIR)

```
});
```

Generación del *script* de CI y CD a través de GitHub Copilot

El *prompt* para GitHub Copilot es «genera un flujo de trabajo de CI y CD para GitHub Actions que haga lo siguiente: instale dependencias de Node.js, ejecute pruebas unitarias con Jest y despliegue la aplicación en un servidor de pruebas mediante SSH».

Archivo de flujo CI y CD (generado por GitHub Copilot)

```
Archivo: .github/workflows/ci_cd.yml

name: CI/CD Pipeline

on:

  push:

    branches:

      - main

  pull_request:

    branches:

      - main

jobs:

  build:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout code
```


© Universidad Internacional de La Rioja (UNIR)

```
      uses: actions/checkout@v2
    - name: Setup Node.js
      uses: actions/setup-node@v2
      with:
        node-version: '16'
    - name: Install dependencies
      run: |
        npm install
    - name: Run Tests
      run: |
        npm test
  deploy:
    runs-on: ubuntu-latest
    needs: build
    steps:
    - name: Checkout code
      uses: actions/checkout@v2
    - name: Setup Node.js
      uses: actions/setup-node@v2
      with:
```


© Universidad Internacional de La Rioja (UNIR)

```
        node-version: '16'

    - name: Install dependencies

      run: |

        npm install

    - name: Deploy to Server

      env:

        SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}

        SERVER_USER: ${{ secrets.SERVER_USER }}

        SERVER_HOST: ${{ secrets.SERVER_HOST }}

      run: |

        echo "${SSH_PRIVATE_KEY}" > private_key.pem

        chmod 600 private_key.pem

        scp -i private_key.pem -r ./* ${{ env.SERVER_USER }}@${{
env.SERVER_HOST }}:/var/www/app

        ssh -i private_key.pem ${{ env.SERVER_USER }}@${{ env.SERVER_HOST
}} "cd /var/www/app &&amp;amp; npm install &&amp;amp; pm2 restart app"
```


© Universidad Internacional de La Rioja (UNIR)

Explicación del archivo CI y CD

Sección	Descripción
On	Define que el flujo de CI y CD se ejecute en los eventos push y `pull_request` hacia la rama `main`.
`jobs.build`	La fase de construcción instala dependencias y ejecuta pruebas unitarias con `Jest`.
`jobs.deploy`	La fase de despliegue despliega la aplicación en un servidor remoto mediante SSH.
`env`	Usa variables de entorno seguras para los secretos (`SSH_PRIVATE_KEY`, `SERVER_USER` y `SERVER_HOST`).

Tabla 2. Explicación del archivo CI y CD. Fuente: elaboración propia.

Configuración de secretos en GitHub

Para permitir que el flujo de despliegue acceda al servidor, es necesario configurar secretos en el repositorio.

- Ve a «Settings» > «Secrets and Variables» > «Actions».
- Añade los siguientes secretos:
  - SSH_PRIVATE_KEY : clave privada SSH para conectarse al servidor de pruebas.
  - SERVER_USER : nombre de usuario del servidor.
  - SERVER_HOST : dirección IP o nombre de *host* del servidor de pruebas.

© Universidad Internacional de La Rioja (UNIR)

Pruebas del flujo CI y CD

- Hacer un `commit` y `push` al repositorio:

```
$> git add .

$> git commit -m "Inicializar flujo CI/CD"

$> git push origin main
```

- Revisión de la ejecución en GitHub Actions:
  - Ve a la pestaña «Actions» en el repositorio de GitHub.
  - Verás que se activa el flujo CI y CD *pipeline*.
  - Se ejecutarán las fases de *build* y *deploy*.
- Despliegue completado. Una vez finalizado el proceso, la aplicación se despliega en el servidor remoto.

Personalización del flujo CI y CD

- Notificaciones de fallos:
  - Se pueden enviar notificaciones por correo o Slack cuando una fase falle.
  - Agregar la siguiente sección al final del flujo:

```
- name: Notify Failure

  if: failure()

  run: |

    echo "Notificación de fallo: La build ha fallado."
```


© Universidad Internacional de La Rioja (UNIR)

- Control de versiones con *tags*. Se puede activar el flujo solo para lanzamientos etiquetados:

```
on:
  push:
    tags:
      - 'v*.*.*'
```

Resumen del proceso

- Generación del *script*. Se usa GitHub Copilot para crear un script YAML de CI y CD.
- Fase de pruebas. El flujo ejecuta pruebas unitarias con `Jest`.
- Despliegue. La aplicación se copia al servidor de pruebas mediante SSH y SCP *(secure copy protocol)*.
- Automatización completa. Cada vez que hay un `push` o `pull request`, se ejecutan las pruebas y se despliega la aplicación.

Conclusiones

- Automatización completa. El flujo de CI y CD generado por GitHub Copilot automatiza la compilación, prueba y despliegue.
- Reducción de errores humanos. Con la automatización, se eliminan los errores manuales de despliegue.
- Mejoras potenciales:
  - Enviar notificaciones en caso de errores.
  - Usar `docker` en lugar de copiar archivos con SCP.

© Universidad Internacional de La Rioja (UNIR)

Ventajas y desventajas del uso de IA en la automatización de scripts

Uso de IA en la automatización de *scripts*			
Ventajas		**Desventajas**	
1	Ahorro de tiempo: la IA acelera la escritura y la optimización de *scripts* repetitivos	1	Dependencia de la IA: los desarrolladores pueden volverse dependientes de las sugerencias de la IA, lo que afecta su capacidad para escribir código sin asistencia
2	Mejora en la calidad del código: reduce errores y permite crear *scripts* más limpios y optimizados	2	Errores no detectados: la IA puede sugerir código que, si bien es funcional, no es siempre el más eficiente o seguro
3	Reducción de la carga cognitiva: los desarrolladores pueden confiar en la IA para gestionar tareas repetitivas, para reservar su enfoque para problemas más complejos	3	Necesidad de validación manual: a pesar de las sugerencias de la IA, es fundamental revisar el código generado para asegurar su calidad

Tabla 3. Ventajas y desventajas. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

3.4. Integración de automatización en flujos de trabajo

La IA está transformando los flujos de trabajo en el desarrollo de *software,* lo que permite **automatizar tareas repetitivas** y **optimizar la eficiencia** de los equipos de desarrollo. La integración de herramientas de IA en estos flujos facilita la ejecución de tareas como **pruebas, despliegues, generación de código y documentación,** lo que les permite a los desarrolladores enfocarse en aspectos más creativos y complejos del proceso de desarrollo.

La integración de IA en los flujos de trabajo permite mejorar la eficiencia, la precisión y la escalabilidad de los proyectos de *software.*

Introducción a la automatización con IA en el desarrollo de software

La **integración de la automatización** en los flujos de trabajo es un proceso esencial en el desarrollo de *software* moderno. Esta práctica busca **optimizar, agilizar y asegurar las tareas** dentro del ciclo de desarrollo mediante el uso de *scripts,* herramientas automatizadas y tecnologías de CI y CD. A través de la automatización, los equipos pueden **reducir el tiempo de desarrollo,** minimizar errores humanos y mejorar la calidad del *software.*

Un flujo de trabajo en el desarrollo de *software* es una secuencia estructurada de pasos o procesos que guían la creación, prueba y despliegue de una aplicación.

© Universidad Internacional de La Rioja (UNIR)

Estos pasos incluyen desde la **escritura de código, pruebas y control de calidad** hasta la entrega de *software* al entorno de producción. Ejemplo de un flujo de trabajo de desarrollo:

- Desarrollo del código.
- Revisión y control de versiones.
- Pruebas unitarias e integración.
- Compilación del *software.*
- Despliegue en servidores o entornos.
- Monitoreo y retroalimentación.

Principales áreas de aplicación

- Pruebas de *software* automatizadas. La IA permite realizar pruebas automatizadas para verificar la funcionalidad, el rendimiento y la seguridad del *software,* lo que detecta posibles errores antes de que lleguen a producción.
- Despliegue continuo (CI y CD). La automatización de flujos de CI y CD permite integrar y desplegar cambios de código en un entorno de producción de manera rápida y sin interrupciones.
- Generación y documentación de código. Hay herramientas de IA, como GitHub Copilot, que generan fragmentos de código y documentan funciones automáticamente, lo que asegura que el código sea comprensible y mantenible.
- Monitoreo de aplicaciones. La IA puede analizar los *logs* de aplicaciones para identificar patrones de errores y posibles fallos, lo que mejora la estabilidad del sistema.

© Universidad Internacional de La Rioja (UNIR)

Ejemplo de integración

- Pruebas automatizadas con GitHub Actions y Copilot. En un flujo de trabajo de CI y CD, cada cambio en el código desencadena una serie de pruebas.
  - Por ejemplo, con GitHub Actions y Copilot, cada vez que un desarrollador envía código se genera automáticamente un conjunto de pruebas unitarias y de integración. Copilot sugiere las pruebas y GitHub Actions ejecuta el *pipeline*.
- Automatización de despliegues en nube con Jenkins e IA. El despliegue en entornos de nube requiere *scripts* de automatización para configurar y escalar servicios.
  - Por ejemplo, Jenkins con un modelo de IA analiza el rendimiento del despliegue en tiempo real, por lo que ajusta la asignación de recursos automáticamente. Si el despliegue afecta el rendimiento de un servicio, el modelo de IA ajusta los parámetros para optimizar el proceso.
- Análisis de *logs* automatizado para depuración en tiempo real con Datadog. Las aplicaciones en producción generan grandes volúmenes de *logs*, que pueden contener indicios de problemas.
  - Por ejemplo, un modelo de IA integrado con herramientas de monitoreo como Datadog analiza los *logs* en tiempo real para detectar patrones de error y alertar al equipo de desarrollo. Esto permite solucionar problemas antes de que afecten a los usuarios.
- AIOps en monitoreo de aplicaciones. Las operaciones asistidas por IA (AIOps) les permiten a las aplicaciones aprender de patrones históricos para prevenir fallos y recomendar configuraciones más estables.

© Universidad Internacional de La Rioja (UNIR)

Caso práctico: análisis de logs automatizado para depuración en tiempo real con Datadog

Contexto

- El objetivo es implementar un sistema automatizado de análisis de *logs* en tiempo real usando Datadog. El sistema detectará patrones de error críticos y enviará alertas automáticas al equipo de desarrollo, antes de que los problemas afecten a los usuarios finales.

- Escenario. Una aplicación web basada en Node.js y desplegada en un servidor en la nube está experimentando errores intermitentes de conexión a la base de datos. Los desarrolladores quieren recibir alertas en tiempo real cada vez que se detecten errores críticos, como ECONNREFUSED , y visualizar estadísticas de los errores en un tablero de control *(dashboard)* de Datadog.

- Requisitos:

  - Monitoreo en tiempo real. Los errores de conexión a la base de datos deben generar una alerta automática.

  - Visualización y *dashboard*. Mostrar estadísticas en un panel de control con el número de errores por minuto.

  - Notificación. Enviar una alerta por correo o Slack al equipo de desarrollo cuando ocurra un error crítico.

© Universidad Internacional de La Rioja (UNIR)

Configuración del entorno

```
Instalar DataDog Agent
```

En el servidor donde se ejecuta la aplicación, instala el Datadog Agent.

```
$> DD_API_KEY=<TU_API_KEY> bash -c "$(curl -L https://s3.amazonaws.com/dd-agent/scripts/install_script.sh)"
```

Configurar el archivo datadog.yaml, para editar el archivo de configuración principal de Datadog (/etc/datadog-agent/datadog.yaml) y habilitar la recopilación de *logs:*

```
logs_enabled: true
```

Para configurar la integración de *logs,* se crea un archivo de configuración de integración de *logs* para Node.js en /etc/datadog-agent/conf.d/nodejs.d/conf.yaml :

```
logs:

  - type: file

    path: /var/log/app.log

    service: app-node

    source: nodejs
```

Código del proyecto Node.js

Instalar dependencias de registro de *logs:*

```
$> npm init -y

$> npm install winston
```

Archivo src/logger.js (gestor de *logs* con Winston):

```
const { createLogger, transports, format } = require('winston');
```


© Universidad Internacional de La Rioja (UNIR)

```
const logger = createLogger({

    level: 'info',

    format: format.combine(

        format.timestamp(),

        format.json()

    ),

    transports: [

        new transports.File({ filename: '/var/log/app.log' }), // Los logs
se almacenan en app.log

        new transports.Console()

    ]

});

module.exports = logger;
```

Archivo src/app.js (simulación de errores en la aplicación):

```
const logger = require('./logger');

function conectarBaseDeDatos() {

    // Simulación de error de conexión

    const error = Math.random() > 0.7; // 30% de probabilidad de error

    if (error) {

        logger.error('Error de conexión a la base de datos: ECONNREFUSED');

    } else {
```


© Universidad Internacional de La Rioja (UNIR)

```
        logger.info('Conexión a la base de datos exitosa');
    }
}
function iniciarAplicacion() {
    logger.info('Iniciando aplicación...');
    setInterval(() => {
        conectarBaseDeDatos();
    }, 5000); // Intenta conectar cada 5 segundos
}
iniciarAplicacion();
```

Configuración de la detección de anomalías con Datadog

Hay que crear una regla de detección de anomalías.

- Ve a «Logs» > «Generate New Monitor» en la consola de DataDog.
- Configura una regla para detectar la siguiente consulta de *log*: service:app-node AND @message:"ECONNREFUSED" .
- Configura la acción de la alerta para enviar notificaciones por correo o Slack.
- Define el límite de tolerancia. Por ejemplo, «si hay más de cinco errores en un minuto, envía una alerta».

© Universidad Internacional de La Rioja (UNIR)

Visualización y *dashboard*

Para crear el panel de control *(dashboard)* en Datadog:

- Ve a «Dashboards» > «Create dashboard».
- Añade *widgets* para:
  - Un gráfico de errores de conexión por minuto (count by @message).
  - Una tabla con los últimos diez errores detectados.

Para configurar las visualizaciones:

- Configura la visualización para mostrar cuántos errores ECONNREFUSED ocurren cada minuto.
- Usa la opción de visualización time series para ver el historial de errores.

Pruebas del sistema

Para generar errores artificiales:

- Corre la aplicación con:

```
$> node src/app.js

Salida de los Logs:

{"level":"info","message":"Iniciando aplicación...","timestamp":"2024-01-
15T10:00:00Z"}

{"level":"error","message":"Error de conexión a la base de datos:
ECONNREFUSED","timestamp":"2024-01-15T10:00:05Z"}

{"level":"info","message":"Conexión a la base de datos
exitosa","timestamp":"2024-01-15T10:00:10Z"}
```


© Universidad Internacional de La Rioja (UNIR)

- Ver los *logs* en Datadog:
  - Ve a *logs* en la consola de Datadog.
  - Filtra por el `service:app-node` y verifica que los errores `ECONNREFUSED` estén siendo capturados.
- Simular un pico de errores:
  - Edita el archivo `src/app.js` y aumenta la probabilidad de error a `Math.random() > 0.2`.
  - Reinicia la aplicación y observa la detección de la anomalía en Datadog.

Resultados y beneficios

Componente	Descripción
Componente	Los errores de conexión a la base de datos se detectan en tiempo real.
Alertas	Se envían alertas por correo o *slack* si se supera el umbral de errores.
Visualización	El equipo puede ver en el *dashboard* cuántos errores han ocurrido en la última hora.
Reducción de tiempo	Se reducen los tiempos de respuesta ante errores críticos.

Tabla 4. Resultados y beneficios. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Automatización con CI y CD

Para asegurar que el sistema esté bien configurado en entornos de desarrollo, producción y pruebas, se puede usar un flujo de CI y CD.

Archivo GitHub Actions (.github/workflows/datadog.yml):

```
name: Deploy with Logs Monitoring

on:

  push:

    branches:

      - main

jobs:

  deploy:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout code

      uses: actions/checkout@v2

    - name: Setup Node.js

      uses: actions/setup-node@v2

      with:

        node-version: '16'

    - name: Install dependencies
```


© Universidad Internacional de La Rioja (UNIR)

```
      run: |
        npm install
    - name: Run Application
      run: |
        node src/app.js &
        sleep 10 # Deja que la app genere algunos logs
    - name: Upload Logs
      env:
        DD_API_KEY: ${{ secrets.DD_API_KEY }}
      run: |
        curl -X POST -H "Content-Type: application/json" \
        -d '{"message": "Despliegue completado con éxito"}' \
        "https://http-intake.logs.datadoghq.com/v1/input/${{
secrets.DD_API_KEY }}"
```


© Universidad Internacional de La Rioja (UNIR)

Conclusiones

- Monitoreo continuo. Los desarrolladores están informados de los errores en tiempo real.
- Alertas automatizadas. Cuando ocurren picos de errores, el equipo recibe notificaciones automáticas.
- Depuración eficiente. Los desarrolladores pueden ver los registros exactos que generaron la alerta.
- Visualización de patrones. Los gráficos permiten ver si los errores ocurren en ciertos horarios o condiciones.

Ventajas y desventajas de la integración de la IA

Las ventajas son:

- La optimización del tiempo, ya que la IA automatiza tareas repetitivas, lo que acelera el ciclo de desarrollo.
- La precisión y la calidad, dado que reduce el riesgo de errores humanos y mantiene la consistencia en el código.
- La capacidad predictiva, puesto que los modelos de IA avanzados pueden predecir problemas en el *pipeline* y mejorar la confiabilidad del *software.*

© Universidad Internacional de La Rioja (UNIR)

Las desventajas son:

- La dependencia de la IA, dado que un uso excesivo de IA en tareas rutinarias puede reducir la capacidad de los desarrolladores para resolver problemas manualmente.
- La complejidad en la integración, puesto que la integración de IA en flujos de trabajo existentes requiere ajustes y puede aumentar la complejidad del sistema.
- Las limitaciones de precisión, ya que la IA no siempre es precisa y, en algunos casos, puede recomendar soluciones incorrectas.

© Universidad Internacional de La Rioja (UNIR)

3.5. Herramientas populares de automatización en desarrollo

La automatización en el desarrollo de *software* es esencial para optimizar procesos como la **compilación, pruebas, despliegue, monitoreo y gestión de entornos.** De esta manera, existen diversas herramientas que facilitan estos procesos y que les permiten a los equipos **mejorar la eficiencia y la calidad del producto.**

A continuación, se presentan algunas de las **herramientas más populares** utilizadas en la automatización del desarrollo de *software,* clasificadas según sus principales funcionalidades.

GitHub Actions

- Funcionalidad. GitHub Actions permite automatizar flujos de trabajo de CI y CD mediante la definición de acciones que se ejecutan en respuesta a eventos en el repositorio. Al ser altamente personalizable, se puede usar para pruebas, despliegues y otros procesos automáticos.

- Aplicaciones en automatización con IA:

  - Despliegue continuo automatizado: con GitHub Actions, se pueden configurar *pipelines* que ejecuten pruebas, desplieguen aplicaciones en entornos de prueba o producción, y generen reportes automáticos.

  - Generación de documentación automática: combina GitHub Actions con herramientas de IA, como Copilot, para generar documentación actualizada del código.

  - Integración con GitHub Copilot: GitHub Actions puede desencadenar *scripts* de Copilot para la generación de pruebas unitarias y de integración de forma automática.

© Universidad Internacional de La Rioja (UNIR)

- Ventaja. GitHub Actions permite la automatización completa de CI y CD sin salir del ecosistema GitHub, lo que facilita la integración y reduce la fricción entre equipos de desarrollo.

Tabnine

- Funcionalidad. Tabnine es un motor de autocompletado de código que usa IA para anticipar y sugerir código en múltiples lenguajes de programación.
- Aplicaciones en automatización:
  - Autocompletado inteligente: sugerencias de código específicas en función de patrones de uso, lo que ayuda a los desarrolladores a escribir código más rápido.
  - Personalización: se adapta a los patrones y estilo de codificación de cada desarrollador, proporcionando sugerencias personalizadas.
- Ventaja. Se integra con numerosos IDE (entornos de desarrollo integrados) y es compatible con más de treinta lenguajes, por lo que ofrece soporte en proyectos de gran envergadura y equipos multilingües.

Jenkins con integración de IA

- Funcionalidad. Jenkins es una herramienta de CI y CD que se puede combinar con modelos de IA para optimizar y monitorear los *pipelines* de desarrollo.
- Aplicaciones en automatización:
  - Optimización de despliegues: la IA puede analizar los *pipelines* de CI y CD para optimizar la velocidad y reducir el riesgo de errores en los despliegues.
  - Monitoreo predictivo: al usar IA, Jenkins puede predecir fallos y tomar acciones preventivas para evitar interrupciones en la producción.

© Universidad Internacional de La Rioja (UNIR)

- Ventaja. Jenkins es una herramienta robusta que, con IA, mejora la eficiencia de los procesos CI y CD y reduce errores en la integración y en el despliegue de nuevas versiones.

DeepCode AI y Snyk Code

- Funcionalidad. DeepCode es una herramienta de análisis de código impulsada por IA que revisa y detecta errores en el código fuente, desde errores de seguridad hasta problemas de rendimiento.

- Aplicaciones en automatización:

  - Detección de vulnerabilidades: identifica problemas de seguridad en tiempo real, mientras que el desarrollador escribe el código.

  - Optimización del código: sugerencias automáticas para mejorar la calidad del código y eliminar posibles errores.

- Ventaja. Se integra con GitHub, GitLab y Bitbucket, y revisa el código en tiempo real para mejorar la seguridad y el rendimiento del *software.*

SonarQube con IA

- Funcionalidad. SonarQube es una plataforma de análisis de código estático que verifica la calidad del código y detecta errores de seguridad, de rendimiento y de estilo.

- Aplicaciones en automatización:

  - Evaluación de calidad: analiza el código y proporciona una puntuación de calidad, por lo que sugiere mejoras en función de estándares de la industria.

  - Monitorización de métricas de código: la IA puede ayudar a identificar patrones de código propensos a errores y proporcionar recomendaciones para mejorar la arquitectura.

© Universidad Internacional de La Rioja (UNIR)

- Ventaja. Con un enfoque en la seguridad y calidad del código, SonarQube ayuda a mantener estándares de codificación altos en proyectos de gran escala.

CodeGuru (Amazon Web Services o AWS)

- Funcionalidad. AWS CodeGuru es un servicio de análisis de código asistido por IA que identifica errores, cuellos de botella de rendimiento y optimiza el código.
- Aplicaciones en automatización:
  - Revisión de código automatizada: CodeGuru analiza el código y proporciona sugerencias de mejora en función de mejores prácticas.
  - Análisis de rendimiento en producción: puede evaluar el rendimiento del código en producción y sugerir ajustes para optimizar la eficiencia de los recursos.
- Ventaja. Al ser un servicio de AWS, se integra fácilmente con otros servicios de Amazon, lo que lo convierte en una opción atractiva para desarrolladores en la nube.

Azure DevOps con IA

- Funcionalidad. Azure DevOps de Microsoft incluye capacidades de IA para optimizar el ciclo de desarrollo, la gestión de proyectos y las pruebas automatizadas.
- Aplicaciones en automatización:
  - Análisis predictivo de *pipelines:* IA integrada para identificar posibles errores y retrasos en los *pipelines* de CI y CD antes de que ocurran.
  - Pruebas automatizadas inteligentes: generación automática de pruebas basadas en el código nuevo o actualizado, lo que mejora la cobertura de pruebas y la confiabilidad del *software.*
- Ventaja. Al integrar IA con las funcionalidades de DevOps, les permite a los equipos gestionar y automatizar tareas de manera efectiva, de manera que mantiene altos estándares de calidad en todo el ciclo de desarrollo.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Google AutoML para pruebas y optimización de modelos

- Funcionalidad. Google AutoML permite crear modelos personalizados sin necesidad de experiencia avanzada en *machine learning,* por lo que es útil para pruebas y optimización en desarrollo.

- Aplicaciones en automatización:

  - Creación de modelos de prueba automatizados: genera modelos personalizados para simular escenarios de pruebas y verificar la funcionalidad del código.

  - Optimización de algoritmos: permite ajustar modelos de IA que puedan integrarse en productos de *software,* lo que optimiza su rendimiento sin intervención manual.

- Ventaja. La personalización de modelos facilita la generación de pruebas automáticas y mejora el rendimiento de algoritmos en diversas aplicaciones.

Harness con IA

- Funcionalidad. Harness es una plataforma CI y CD que integra IA para supervisar el proceso de despliegue continuo, de manera que ajusta los flujos de trabajo automáticamente.

- Aplicaciones en automatización:

  - Verificación automática de despliegues: la IA de Harness evalúa los despliegues en producción y puede revertir cambios si detecta anomalías.

  - Asignación de recursos inteligente: ajusta automáticamente los recursos según la demanda, por lo que optimiza el rendimiento y reduce los costos.

- Ventaja. Mejora la estabilidad de los despliegues en producción y reduce el riesgo de errores críticos.

© Universidad Internacional de La Rioja (UNIR)

Datadog con IA para monitorización de aplicaciones

- Funcionalidad. Datadog es una plataforma de monitorización que usa IA para analizar el rendimiento de aplicaciones, identificar anomalías y generar alertas en tiempo real.
- Aplicaciones en automatización:
  - Análisis predictivo de errores: utiliza IA para detectar patrones anormales en el rendimiento de aplicaciones y anticipar posibles problemas antes de que ocurran.
  - Alertas automatizadas: genera alertas automáticas basadas en análisis de patrones históricos y en tiempo real.
- Ventaja. Mejora la confiabilidad de las aplicaciones en producción y facilita la detección temprana de errores, de manera que optimiza el tiempo de respuesta.

© Universidad Internacional de La Rioja (UNIR)

Herramienta	Aplicación principal	Ventaja
GitHub Actions	Automatización de CI y CD, despliegues y pruebas	Permite la automatización completa dentro del ecosistema GitHub, lo que simplifica los flujos de trabajo
GitHub Copilot	Autocompletado y generación de pruebas	Ahorra tiempo y mejora la documentación
Tabnine	Autocompletado en múltiples lenguajes	Compatible con muchos IDE y lenguajes
Jenkins + IA	Optimización de CI y CD	Mejora la eficiencia de despliegues
DeepCode AI/Snyk Code	Análisis de código y detección de errores	Mejora seguridad y rendimiento
SonarQube	Análisis de calidad de código	Mantiene altos estándares de calidad
CodeGuru	Revisión de código y optimización en AWS	Ideal para desarrollos en la nube AWS
Azure DevOps	Pruebas automáticas y gestión de CI y CD	Administración eficiente del ciclo de desarrollo
AutoML	Generación de modelos personalizados	Facilita pruebas automáticas complejas
Harness	Despliegues continuos con verificación automática	Aumenta la estabilidad en producción
Datadog	Monitorización de aplicaciones	Detecta problemas antes de que afecten a usuarios

Tabla 5. Resumen. Fuente: elaboración propia.

Estas herramientas de IA son recursos muy utilizados para automatizar tareas en el desarrollo de *software,* ya que incrementan la productividad, reducen errores y optimizan el rendimiento de los equipos.

© Universidad Internacional de La Rioja (UNIR)

3.6. Mejora de eficiencia y reducción de errores con automatización

La IA está transformando el desarrollo de *software* al **optimizar cada etapa del proceso,** desde la planificación y codificación hasta las pruebas y despliegue. Gracias a herramientas de IA avanzadas, los desarrolladores pueden **automatizar tareas,** reducir errores y **mejorar la eficiencia** en cada fase del ciclo de vida de desarrollo.

Planificación y diseño

La etapa de planificación y diseño se enfoca en definir los requisitos del proyecto, diseñar la arquitectura del sistema y planificar los *sprints* y las tareas de desarrollo.

- Análisis de requisitos asistido por IA:
  - Hay herramientas, como IBM Watson y OpenAI GPT-4-Turbo, que ayudan a analizar requisitos y a realizar una extracción de entidades clave a partir de documentos y correos electrónicos.
  - Ventaja: reducen el tiempo de análisis y ayudan a evitar errores de interpretación de requisitos.
  - Ejemplo práctico: GPT-4-Turbo se puede usar para generar una lista detallada de requisitos a partir de una descripción de proyecto, lo que facilita la identificación de dependencias o requisitos ambiguos.

© Universidad Internacional de La Rioja (UNIR)

- Generación automática de diagramas de arquitectura:
  - Hay herramientas de IA, como Lucidchart AI, que crean diagramas de arquitectura basados en descripciones de alto nivel, por lo que agilizan el diseño inicial.
  - Ventaja: permite visualizar la arquitectura rápidamente y mejora la comprensión del proyecto.
  - Ejemplo práctico: al ingresar una descripción del sistema, la IA genera un diagrama inicial que el equipo de desarrollo puede ajustar.

Codificación y autocompletado

La codificación es la **fase central del desarrollo,** ya que se traduce el diseño en código funcional. Además, hay herramientas de IA que ayudan a mejorar la eficiencia de los desarrolladores, a **reducir errores comunes** y a **aumentar la velocidad de escritura de código.**

- Autocompletado de código:
  - Hay herramientas, como GitHub Copilot y Tabnine, que sugieren fragmentos de código en tiempo real, mientras que el desarrollador escribe, lo que reduce el tiempo de escritura y mejora la consistencia.
  - Ventaja: acelera la codificación al sugerir estructuras completas, funciones y variables basadas en el contexto.
  - Ejemplo práctico: al escribir una función de procesamiento de datos en Python, Copilot puede sugerir automáticamente bucles y funciones comunes para manipulación de datos, lo que reduce el esfuerzo del desarrollador.
- Corrección de errores sintácticos y lógicos:
  - DeepCode y Snyk analizan el código en tiempo real, por lo que identifican errores de sintaxis, vulnerabilidades de seguridad y problemas lógicos.

© Universidad Internacional de La Rioja (UNIR)

- Ventaja: minimiza el riesgo de errores en etapas posteriores del desarrollo.
- Ejemplo práctico: DeepCode detecta una posible vulnerabilidad de inyección de SQL *(structured query language)* en el código y sugiere una solución segura.

Pruebas y validación automática

La etapa de pruebas es esencial para asegurar que el código funcione correctamente y que cumpla con los requisitos. Las herramientas de IA permiten automatizar pruebas, mejorar la cobertura y detectar errores en etapas tempranas.

- Generación de pruebas unitarias y de integración:
  - Hay herramientas, como GitHub Copilot, que pueden sugerir pruebas unitarias automáticas para cada función, mientras que CodeT5 genera pruebas de integración para validar módulos completos.
  - Ventaja: aumenta la cobertura de pruebas y reduce la carga manual de crear pruebas para cada componente.
  - Ejemplo práctico: al escribir una función en JavaScript, Copilot sugiere automáticamente una prueba unitaria en Jest para asegurar que la función devuelva el valor esperado.
- Automatización de pruebas funcionales y de carga:
  - Selenium con IA y LoadNinja permiten realizar pruebas automatizadas de interfaz y pruebas de carga, dado que detectan errores y aseguran el rendimiento bajo condiciones de alta demanda.
  - Ventaja: reduce el tiempo de prueba y asegura que la aplicación cumpla con los requisitos de rendimiento antes del lanzamiento.

© Universidad Internacional de La Rioja (UNIR)

- Ejemplo práctico: LoadNinja simula miles de usuarios simultáneos para una aplicación web y analiza el rendimiento del servidor.

- Análisis de *logs* para identificación de errores:

- Datadog y Splunk con IA analizan registros en tiempo real para identificar patrones de error, por lo que ayudan a los equipos a solucionar problemas de rendimiento.

- Ventaja: proporciona visibilidad del rendimiento en tiempo real y ayuda a detectar problemas antes de que afecten a los usuarios.

- Ejemplo práctico: Datadog detecta una caída en el rendimiento durante una operación de base de datos y envía una alerta al equipo de desarrollo para revisar el problema.

Despliegue e integración continua

El CI y el CD son claves para llevar el *software* a producción de forma segura y sin interrupciones. La IA permite optimizar estos procesos, ya que reduce tiempos de espera y errores en producción.

- Automatización del *pipeline* de CI y CD:

- GitHub Actions y Jenkins con IA automatizan el *pipeline* de CI y CD, por lo que integran pruebas automáticas y optimizan el despliegue en producción.

- Ventaja: permiten un flujo de despliegue rápido y reducen los errores de integración.

- Ejemplo práctico: con GitHub Actions, cada vez que un desarrollador hace un push a la rama principal el sistema ejecuta pruebas y despliega automáticamente la nueva versión en el entorno de pruebas.

© Universidad Internacional de La Rioja (UNIR)

- Optimización de recursos en despliegue:
  - AWS CodeGuru y Azure DevOps ajustan automáticamente la asignación de recursos en el despliegue según la carga esperada, por lo que optimizan el rendimiento de la aplicación en la nube.
  - Ventaja: asegura un rendimiento óptimo en producción y reduce costos en entornos de nube.
  - Ejemplo práctico: CodeGuru ajusta los parámetros de memoria y de CPU *(central processing unit)* en una aplicación desplegada en AWS Lambda, para mejorar su eficiencia sin afectar el rendimiento.

Mantenimiento y monitorización en producción

Una vez en producción, el *software* necesita **mantenimiento continuo y monitorización** para asegurar su correcto funcionamiento. Hay herramientas de IA que permiten **monitorizar el rendimiento** y **realizar ajustes proactivos** en el sistema.

- Monitorización en tiempo real:
  - Herramientas como Datadog y New Relic usan IA para analizar el rendimiento en tiempo real y detectar anomalías antes de que afecten a los usuarios.
  - Ventaja: permiten solucionar problemas antes de que se conviertan en incidentes críticos, por lo que mejoran la experiencia del usuario.
  - Ejemplo práctico: Datadog detecta un aumento en los tiempos de respuesta durante una operación en la base de datos y alerta al equipo de soporte para realizar ajustes.
- Optimización de recursos con IA:
  - Harness y Dynatrace ajustan la asignación de recursos en función del uso, por lo que escalan automáticamente los servicios para adaptarse a la demanda.

© Universidad Internacional de La Rioja (UNIR)

- Ventaja: mejoran el rendimiento y aseguran la disponibilidad de la aplicación, de manera que optimizan costos.

- Ejemplo práctico: Dynatrace escala automáticamente los recursos de una aplicación web en AWS durante picos de tráfico, por lo que asegura la disponibilidad.

Etapa	Herramientas principales	Aplicación en eficiencia y reducción de errores
Planificación y diseño	IBM Watson, Lucidchart AI y GPT-4-Turbo	Análisis de requisitos y generación de diagramas de arquitectura
Codificación	GitHub Copilot, Tabnine y DeepCode y Snyk	Autocompletado y detección de errores de seguridad
Pruebas y validación	Selenium, LoadNinja, GitHub Copilot y CodeT5	Generación de pruebas automáticas y pruebas de carga
Despliegue e integración	GitHub Actions, Jenkins + IA y AWS CodeGuru	CI y CD automatizado, y optimización de recursos en despliegue
Mantenimiento y monitorización	Datadog, Dynatrace, New Relic y Harness	Monitorización en tiempo real y escalado automático de recursos

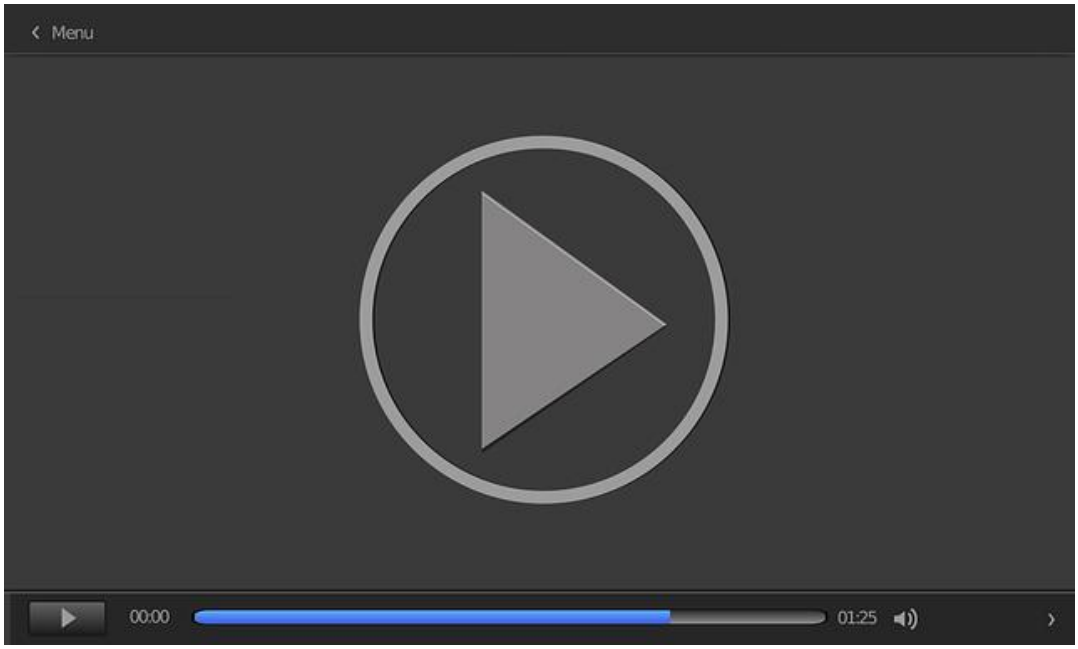
Tabla 6. Tabla resumen. Fuente: elaboración propia.

La incorporación de IA en las diversas etapas del desarrollo de *software* permite **mejorar la eficiencia** y **reducir errores significativamente.** Desde la planificación hasta la monitorización en producción, la IA optimiza el flujo de trabajo, por lo que asegura que el *software* sea de **alta calidad, seguro y eficiente** en cada etapa del ciclo de vida del desarrollo. Esto les permite a los equipos de desarrollo enfocarse en tareas de mayor valor y reducir el tiempo dedicado a actividades repetitivas o propensas a errores.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A continuación, veremos el vídeo *Prompt Engineering para generación de código.*



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=07cdb526-aba1-4867-b362-b21801018b5b

© Universidad Internacional de La Rioja (UNIR)

3.7. Referencias bibliográficas

Bryxx. (s. f.). *It's a dirty job, but you can automate it.* https://bryxx.eu/dirty-job-can-automate/

BuzzyBrains. (s. f.). *DevOps – The Complete Guide.* https://www.buzzybrains.com/blog/complete-guide-to-devops/

© Universidad Internacional de La Rioja (UNIR)

Centro de aprendizaje DataDog

Página web de Datadog Learning Center. https://learn.datadoghq.com/

En el centro de cursos y certificaciones de Datadog, se puede aprender todo sobre las características y uso de esta herramienta, especialmente sobre el monitoreo de *software.*

© Universidad Internacional de La Rioja (UNIR)

Guías para GitHub Actions

GitHub Docs. (s. f.). *Guides for GitHub Actions.* https://docs.github.com/en/actions/guides

Conjunto de guías y tutoriales para entender y aprender a desarrollar acciones automatizadas sobre proyectos de *software* en GitHub.

© Universidad Internacional de La Rioja (UNIR)

Snyk Code

Snyk User Docs. (s. f.). *Snyk Code.* https://docs.snyk.io/scan-with-snyk/snyk-code

Documentación oficial de Snyk Code para aprender a utilizar todos los recursos de la herramienta para el análisis y la detección de vulnerabilidades *software.*

© Universidad Internacional de La Rioja (UNIR)

From zero to hero

Edwards A. y Janashia N. (2022, octubre 31). *Azure DevOps: Tutorial de cero a héroe.* https://learn.microsoft.com/es-es/shows/devops-lab/azure-devops-zero-to-hero-tutorial

En este vídeo, desarrollado por los desarrolladores de Azure DevOps, se explican todas las funcionalidades y las ventajas de utilizar esta plataforma, así como la integración con los *pipelines* de desarrollo y el uso de sus herramientas de IA.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cuál es el objetivo principal de la automatización en desarrollo de *software*?

   A. Reducir costos.

   B. Optimizar los procesos y minimizar los errores humanos.

   C. Aumentar el número de líneas de código.

   D. Eliminar la necesidad de pruebas.

2. ¿Qué es una tarea repetitiva en desarrollo de *software*?

   A. Una tarea que se realiza solo una vez.

   B. Una tarea no susceptible a errores.

   C. Una tarea frecuente y predecible, propensa a errores si es manual.

   D. Una tarea que requiere intervención creativa.

3. Una ventaja de usar *scripts* automatizados es:

   A. Aumentar los errores humanos.

   B. Mejorar la productividad y reducir los errores.

   C. Eliminar la documentación del código.

   D. Requerir intervención manual en cada tarea.

4. ¿Cuál de las siguientes es una herramienta para automatización en CI y CD?

   A. Photoshop.

   B. GitHub Actions.

   C. Excel.

   D. Slack.

© Universidad Internacional de La Rioja (UNIR)

5. GitHub Copilot ayuda en la automatización de tareas de desarrollo al:

   A. Generar código automáticamente.

   B. Monitorear los tiempos de entrega.

   C. Crear pipelines de despliegue.

   D. Organizar reuniones.

6. ¿Qué tarea se puede automatizar con Selenium?

   A. Creación de gráficos de diseño.

   B. Pruebas de interfaz de usuario.

   C. Escritura de documentos de requisitos.

   D. Envío de correos electrónicos.

7. La función principal de Tabnine es:

   A. Ayudar en la gestión de proyectos.

   B. Sugerir fragmentos de código y completar código automáticamente.

   C. Generar gráficos y visualizaciones.

   D. Crear informes de ventas.

8. ¿Cuál de las siguientes herramientas facilita la integración continua?

   A. PowerPoint.

   B. Jenkins.

   C. OneNote.

   D. Illustrator.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Para qué se utiliza DeepCode en la automatización de desarrollo?

A. Para crear interfaces de usuario.

B. Para detectar errores de seguridad y optimizar el código.

C. Para organizar reuniones de equipo.

D. Para hacer gráficos.

10. Una ventaja de usar IA en la automatización de pruebas es:

A. Evitar completamente las pruebas manuales.

B. Reducir el tiempo de prueba y mejorar la cobertura.

C. Eliminar la necesidad de control de versiones.

D. Desplegar el *software* sin pruebas.

11. ¿Qué función cumple GitHub Actions en un flujo de trabajo CI y CD?

A. Permitir escribir documentos.

B. Automatizar pruebas, despliegues y generación de informes.

C. Generar contenido gráfico.

D. Desarrollar aplicaciones móviles.

12. Datadog es útil para:

A. Producir informes de ventas.

B. Monitorear aplicaciones y detectar anomalías.

C. Organizar reuniones.

D. Crear presentaciones.

© Universidad Internacional de La Rioja (UNIR)

13. Una ventaja de la automatización en pruebas de *software* es:

A. Disminuir la cobertura de pruebas.

B. Aumentar los errores humanos.

C. Asegurar consistencia en pruebas repetitivas.

D. Aumentar la intervención manual.

14. ¿Qué herramienta se utiliza para monitorización predictiva de CI y CD?

A. Photoshop.

B. Jenkins con IA.

C. PowerPoint.

D. Notepad.

15. La ventaja de usar herramientas de IA en DevOps es:

A. Aumentar la complejidad del código.

B. Facilitar la colaboración y monitoreo del rendimiento.

C. Disminuir la cobertura de pruebas.

D. Aumentar el tiempo de pruebas.

16. ¿Qué permite GitLab CI/CD?

A. Generar gráficos financieros.

B. Automatizar pruebas y despliegues de código.

C. Organizar archivos en la nube.

D. Crear aplicaciones móviles.

© Universidad Internacional de La Rioja (UNIR)

17. Terraform se utiliza principalmente para:

A. Generación de documentación.

B. Gestionar infraestructura como código.

C. Crear *scripts* de migración de datos.

D. Desarrollar interfaces gráficas.

18. Un beneficio de usar Harness en despliegues es:

A. Requerir monitoreo constante del usuario.

B. Incrementar el uso de recursos.

C. Verificación automática y escalado de recursos.

D. Realizar despliegues manuales.

19. Una herramienta para realizar pruebas de carga automatizadas es:

A. Illustrator.

B. LoadNinja.

C. Excel.

D. Slack.

20. La herramienta CodeGuru es conocida por:

A. Generar gráficos interactivos.

B. Revisión de código y optimización de rendimiento en AWS.

C. Automatizar pruebas de interfaz gráfica.

D. Crear documentos de requisitos.

© Universidad Internacional de La Rioja (UNIR)